Parser suitability report

Which parser works best for which kind of document, based on the parsers in this POC.

How this was measured (and how far to trust it)

Evidence	What was done	Strength
A. Scored test set	8 small synthetic documents (digital/scanned/mixed PDF, docx, pptx, xlsx, csv, png), each with a hand-written correct answer. Every free parser was scored on text, headings and tables (`python scripts/evaluate.py`, results in `eval/results.md`).	Objective, but documents are tiny and clean, so many parsers tie.
C. UI runs with answer keys	Seven PDFs run through the web UI and scored against a hand-written reference: page 1 of a research paper, the full 15-page paper, two Wikipedia articles (6 and 3 pages), an internal how-to guide made mostly of screenshots (6 pages) a documentation web page saved as PDF (4 pages), and a clean scanned copy (pages turned into pictures) of the 3-page AI agent article. See "Results by document".	Quantitative, but only 6 different documents so far (the research paper is used twice), and only one is a scan.
B. Real documents	8 real files (a 2-column research paper, a 33-page notes PDF, a clean business PDF, a mixed PDF, 3 screenshots/ID photo, an Excel sheet). Only the first 2 pages were used for PDFs. No answer key, so outputs were read and judged by eye.	Realistic, but qualitative.

Not tested: Azure Document Intelligence, LlamaParse (no API keys) and the vision LLM (Ollama not running).

Treat the table below as a well-supported starting recommendation, not a final decision. The missing piece is 3-5 real documents per type with correct answers (see "Next steps").

Recommendation per document type

Document type	Primary	Backup	Avoid	Why
Digital PDF, simple (one column, mostly text)	**pymupdf**	**hybrid_pdf** (same result here), pymupdf4llm	pdfminer, pdfplumber, markitdown	Two Wikipedia articles: pymupdf scored 99% and 100% in 1.3-1.6 s, the same as docling (99% and 100% in 75 s and 26 s). pdfminer, pdfplumber and markitdown got the words right but no headings (50%).
Digital PDF, complex (two columns, tables)	**pymupdf4llm** (90% in 14 s)	**docling** (95% in 207 s, use only if time does not matter)	pymupdf, hybrid_pdf, pdfminer, pdfplumber, markitdown	Full 15-page research paper: pymupdf fell to 54% (tables 12%), pymupdf4llm kept tables at 80%. Page 1 alone had looked fine for pymupdf (92%), so a single page can mislead.
Digital PDF made of screenshots (little typed text)	**pymupdf** for the typed text	**hybrid_pdf** (same result)	docling, pymupdf4llm, ocr, easyocr if you only need the typed text	Internal how-to guide, 6 pages: pymupdf and hybrid_pdf 100% in 1.5 s. docling 85% (57.6 s) missed part of the typed text (217 of 266 words). pymupdf4llm 58%, ocr 22%, easyocr 16%: they also read the text inside the screenshots (888 to 1,293 words), which the reference does not include. If the screenshot text matters, this test did not measure it.
Scanned PDF (clean scan)	**pymupdf4llm** (93% in 21 s)	**docling** (98% in 59 s, best quality, slowest) ; **ocr** or **hybrid_pdf** if only plain text is needed (97% text in 10-14 s, no headings)	pymupdf, pdfminer, pdfplumber, markitdown, opendataloader	Clean scan of a 3-page article: docling 98%, pymupdf4llm 93%, unstructured 92%. ocr, hybrid_pdf and easyocr read the words (97%) but give no headings, so they score 49%. Five parsers returned nothing (0%) because there is no text layer. Only one clean scan has been tested; a tilted, noisy scan and a mixed file are still to be run.
Mixed PDF (digital + scanned pages)	**pymupdf4llm**	**hybrid_pdf**	pymupdf alone	A and B: pymupdf silently dropped the scanned page (1 empty page). pymupdf4llm scored 1.0; hybrid_pdf 0.89 (scanned page came back without headings).

Images / screenshots / photos	**easyocr** (more accurate)	**ocr** (3-18x faster here, noisier)	docling for screenshots	B: on the Docker page and Windows screenshot, easyocr read UI text most cleanly; docling dropped text on the Windows screenshot (51 words vs ~120) and left `<!-- image -->` placeholders.
ID cards / forms in photos	*No good free option found*	easyocr / docling	ocr	B: every free engine produced partly garbled output on the ID photo (mixed-script text, misread characters). Azure DI or the vision LLM should be tested here.
Word (.docx)	**docx** (native)	docling	markitdown	A: docx, docling, unstructured scored 1.0; markitdown 0.89 (lost heading levels).
PowerPoint (.pptx)	**pptx** (native)	docling	markitdown	A: pptx 0.96, docling 0.91, markitdown 0.77.
Excel / CSV	**spreadsheet** (native, 0.1 s)	markitdown	-	A: all scored near 1.0. B: spreadsheet kept sheet headings and both tables on the real workbook; docling lost the sheet heading.

Results by document

Scores are from the web UI, compared with a hand-written reference. Score = average of text, headings and tables (a "-" means the reference has none of that, so it is not counted).

1. Research paper, page 1 only (`1706.03762v1.pdf`, digital, 1 page)

Parser	Score	Text	Headings	Time
docling	98%	96%	100%	101 s
hybrid_pdf	92%	99%	86%	0.3 s
pymupdf	92%	99%	86%	0.4 s
pymupdf4llm	88%	97%	80%	2.8 s
opendataloader	71%	98%	43%	3.4 s
unstructured	60%	94%	25%	13.4 s
ocr	50%	99%	0%	2.3 s
easyocr	49%	97%	0%	73.3 s
pdfminer	47%	94%	0%	0.1 s
pdfplumber	20%	40%	0%	0.1 s
markitdown	8%	17%	0%	3.2 s

Note: markitdown read the whole 15-page file even though only page 1 was asked for, which explains its very low score.

2. Research paper, full 15 pages (`1706.03762v1.pdf`, digital, two columns, 4 tables)

Parser	Score	Text	Headings	Tables	Time
docling	95%	98%	94%	93%	207 s
pymupdf4llm	90%	94%	94%	80%	13.9 s

opendataloader	56%	96%	67%	5%	4.8 s
pymupdf	54%	95%	56%	12%	6.3 s
hybrid_pdf	54%	95%	56%	12%	6.6 s
unstructured	42%	97%	28%	0%	2.8 s
pdfminer	33%	98%	0%	0%	2.3 s
markitdown	33%	37%	0%	60%	3.1 s
easyocr	32%	96%	0%	0%	440 s
ocr	32%	95%	0%	0%	38.1 s
pdfplumber	13%	37%	0%	2%	6.3 s

Winner: docling for accuracy, pymupdf4llm for accuracy per second. The router's pick (pymupdf) scored only 54% here.

3. Wikipedia article "Generative AI", pages 1-6 (`Generative_AI_p1-6.pdf`, digital, one column, no tables)

Parser	Score	Text	Headings	Time
docling	99%	99%	100%	74.9 s
pymupdf	99%	100%	97%	1.3 s
hybrid_pdf	99%	100%	97%	1.5 s
pymupdf4llm	95%	91%	100%	12.0 s
opendataloader	92%	100%	84%	2.4 s
unstructured	87%	100%	75%	1.0 s
pdfminer	50%	100%	0%	0.5 s
markitdown	50%	100%	0%	1.0 s
pdfplumber	50%	100%	0%	2.3 s
easyocr	48%	96%	0%	477 s
ocr	48%	96%	0%	54.5 s

Winner: pymupdf (same score as docling, 58 times faster). Notes: pymupdf4llm's text score is lower partly because it also read text inside the figures (2,430 words against 2,188 in the reference); the reference does not include picture text. markitdown returned all 6 pages as one page, so page numbers are lost.

4. Wikipedia article "AI agent", pages 1-3 (`AI_agent_p1-3.pdf`, digital, one column, bullet lists, no tables)

Parser	Score	Text	Headings	Time
hybrid_pdf	100%	100%	100%	1.5 s
pymupdf	100%	100%	100%	1.6 s

docling	100%	100%	100%	26.0 s
unstructured	98%	100%	95%	1.9 s
opendataloader	97%	100%	95%	3.5 s
pymupdf4llm	95%	96%	95%	3.1 s
pdfminer	50%	100%	0%	0.7 s
pdfplumber	50%	100%	0%	1.5 s
markitdown	50%	100%	0%	2.4 s
ocr	49%	97%	0%	9.0 s
easyocr	49%	97%	0%	178.7 s

Winner: pymupdf (or hybrid_pdf, same output). The reference has 10 headings; pymupdf, hybrid_pdf and docling found exactly 10, pymupdf4llm and opendataloader found 9, unstructured found 11. The bullet lists were not scored separately (the scorer has no list check yet). markitdown again returned all pages as one page.

5. Internal how-to guide, 6 pages (digital PDF, mostly screenshots with short typed steps, no tables)

The reference contains only the typed text (266 words). Text inside the screenshots is not in it, so parsers that read pictures (OCR) collect many extra words and lose points on text. This test measures how well each parser reads the typed text of a screenshot-heavy PDF.

Parser	Score	Text	Headings	Words found	Time
pymupdf	100%	100%	100%	268	1.5 s
hybrid_pdf	100%	100%	100%	268	1.6 s
docling	85%	78%	92%	217	57.6 s
opendataloader	81%	87%	75%	255	2.2 s
unstructured	79%	100%	59%	281	0.6 s
pymupdf4llm	58%	30%	86%	1,031	31.9 s
pdfminer	50%	100%	0%	262	0.3 s
pdfplumber	50%	100%	0%	262	0.3 s
markitdown	50%	100%	0%	262	0.7 s
ocr	22%	45%	0%	888	11.2 s
easyocr	16%	32%	0%	1,293	305 s

Winner: pymupdf (or hybrid_pdf). The analyzer called this file "digital" because every page has some typed text, even though most of each page is a picture.

6. Documentation web page saved as PDF (`Overview of Amazon Web Services`, 4 pages, digital, menus and footer links around a short article)

The reference contains only the article (271 words). The PDF text has about 660 word tokens, because the left menu, the right-hand links, the footer and many long web addresses are in it too. Every parser read the article words, and also the menus, so the text score is about 58% for almost all of them. The scores below mainly show how many extra "headings" each parser invented from menu items (the reference has 2 headings).

Parser	Score	Text	Headings	Headings found	Time
pymupdf4llm	47%	58%	36%	9	3.8 s
hybrid_pdf	46%	58%	33%	10	1.3 s
pymupdf	46%	58%	33%	10	1.4 s
docling	46%	41%	50%	6	31.8 s
opendataloader	42%	58%	25%	14	2.0 s
unstructured	31%	57%	4%	96	0.9 s
ocr	29%	59%	0%	0	6.7 s
easyocr	29%	58%	0%	0	172 s
pdfplumber	29%	58%	0%	0	0.9 s
pdfminer	28%	57%	0%	0	0.7 s
markitdown	28%	57%	0%	0	1.0 s

Result: a tie in practice (47% to 46% for the top four). This test does not separate the parsers. What it shows: no parser removes the menus and footer from a web page saved as PDF, so that cleaning has to be done separately if only the article is wanted. unstructured found 96 headings, so it is a poor choice when headings matter. docling's lower text score (41%) was not investigated.

7. Scanned copy of the "AI agent" article (`AI_agent_scanned_clean.pdf`, 3 pages, every page a picture, no tables)

The reference is the same as for the digital version (1,215 words, 10 headings). The pages were rendered at 150 dpi and saved as a picture-only PDF. The analyzer classified it as "scanned".

Parser	Score	Text	Headings	Words found	Time
docling	98%	96%	100%	1,202	58.9 s
pymupdf4llm	93%	95%	91%	1,161	21.3 s
unstructured	92%	97%	87%	1,237	25.9 s
easyocr	49%	97%	0%	1,202	160.8 s
hybrid_pdf	49%	97%	0%	1,221	9.6 s
ocr	49%	97%	0%	1,221	13.5 s
pdfminer	0%	0%	0%	0	0.0 s
pdfplumber	0%	0%	0%	0	0.0 s
markitdown	0%	0%	0%	0	0.1 s

pymupdf	0%	0%	0%	0	0.5 s
opendataloader	0%	0%	0%	0	1.3 s

Winner: docling for quality, pymupdf4llm for quality per second. The router's current pick for scans (ocr) read the words well but lost all headings, so it scored 49%. hybrid_pdf gave exactly the same output as ocr, because for picture pages it runs OCR. The five parsers that only read a text layer returned nothing. The Wikipedia logo on page 1 is visible in the scan but is not in the reference, which lowers the text score by about 1 point for the OCR parsers.

What the results show together

- On simple text PDFs (both Wikipedia articles) pymupdf and hybrid_pdf scored 99-100% in about 1.5 s, and any text reader gets the words right. The difference is in **headings** (pdfminer, pdfplumber, markitdown give none) and **tables**.
- On the complex paper the gap is large. pymupdf lost tables and headings; only docling and pymupdf4llm kept them.
- On the screenshot-heavy guide, parsers that run OCR on pictures (pymupdf4llm, docling, ocr, easyocr) returned 4 to 5 times more words than the typed text and scored lower. Whether that extra text is wanted depends on the use case.
- A web page saved as PDF contains menus, footers and long URLs. All text parsers keep them, so the score depends mostly on how the reference is written, not on the parser. Do not use such a document to pick a winner; use it to see what extra text comes along.
- On a scan, the OCR-only parsers (ocr, easyocr, hybrid_pdf) read the words as well as the others (97%) but give no headings. The parsers that analyse the page layout (docling, pymupdf4llm, unstructured) also recover headings.
- docling is the most accurate but 30-250 times slower than pymupdf. OCR engines (ocr, easyocr) are slow and give no headings or tables, so they are only for pictures.

Things worth knowing

1. **Never judge a parser by word count.** On the notes PDF, markitdown reported 3,905 words for 2 pages (other parsers: ~180) which suggests it ignored the 2-page limit (not confirmed). Its output was also fragmented (`TTrraaiinn WWiitthh`).
2. **Text correct is not enough.** pdfminer and pdfplumber got the words right but flattened or broke tables. pdfminer wrote the sales table column by column (`Region North South West Q1 120 98 75...`), so you can no longer tell which number belongs to which row.
3. **Some PDFs have "fake bold" text** (letters printed twice). pymupdf and hybrid_pdf turned this into duplicate headings (`# Train Train`); pymupdf4llm cleaned it up. pdfplumber doubled the letters instead.
4. **Speed differs a lot.** docling took 40-90 s for 1-2 pages on this CPU machine, easyocr 15-60 s, pymupdf 2-12 s, spreadsheet/docx/pptx under a second. Slow engines should be a backup or a "prefer quality" option, not the default.
5. **OCR does not produce headings.** ocr/easyocr return plain text, so they score 0 on headings by nature. If heading structure matters for scans, use pymupdf4llm or docling.

What this means for the router (`app/routing/router.py`)

The current rules are mostly consistent with the data. Suggested changes:

- Digital PDF: pymupdf is enough for simple text PDFs, but it is weak on tables and two-column layouts. The analyzer already estimates the number of tables per PDF (`est_tables`), so a rule like "tables found -> pymupdf4llm first, otherwise pymupdf first" is worth trying. Test it on more documents first.

- Mixed PDF: put **pymupdf4llm** ahead of `hybrid_pdf`, keep `ocr` as last resort.
- Images: consider **easyocr** before `ocr` when accuracy matters more than speed.
- Scanned PDF: the current rule puts `ocr` first. It reads the words but returns no headings. Try `pymupdf4llm` first (93% in 21 s on the test scan), and keep `ocr` as the fast fallback.
- Keep docx / pptx / spreadsheet native-first rules as they are.
- pdfminer, pdfplumber and markitdown are not recommended as a first choice for PDFs.

Next steps to make this final

1. Collect 3-5 real documents per type (invoices, scanned forms, multi-column reports, tables, ID photos).
2. For each, save the correct content as `eval/<file>.truth.md` (Markdown: `#` headings, `|` tables).
3. Run `python scripts/evaluate.py`; the best parser per type is in `eval/results.md`.
4. Test Azure DI / LlamaParse / vision LLM on the types marked "no good free option".

To look at any document interactively: start the app (`uvicorn app.main:app --port 8000`), open http://localhost:8000, choose a file, paste the correct text under "Reference text", and click "Compare all parsers".